

Bynry Automation Engineering Intern – Case Study –
Nirmal Chaturvedi

Bynry Automation Engineering Intern

QA Automation Engineering Intern – Case Study

Candidate Name: Nirmal Chaturvedi

Email: nirmalch1004@gmail.com

Submission Date: 4 August 2026

Table of Contents

1. Introduction
2. Part 1 – Debugging Flaky Tests
3. Part 2 – Test Automation Framework
4. Part 3 – API + UI Integration Testing
5. Assumptions
6. Future Improvements
7. Conclusion

Introduction

This document presents my solution for the QA Automation Engineering Intern Case Study. The objective is to demonstrate my understanding of software testing, automation framework design, API testing, Playwright automation, and testing strategies for a multi-tenant SaaS application.

Where requirements are intentionally incomplete, I have clearly stated my assumptions and explained the reasoning behind my decisions. My focus is not only on writing test cases but also on building maintainable, scalable, and reliable automation.

Part 1 – Debugging Flaky Playwright Tests

Overview

The case study provides two Playwright test cases for login functionality and multi-tenant access verification. Although the tests appear correct at first glance, they are unreliable because they do not properly handle asynchronous page loading, dynamic content, or differences between local and CI/CD execution environments. The objective of this section is to identify the causes of flakiness, explain why they occur, and propose a more reliable automation approach.

What is a Flaky Test?

A flaky test is an automated test that sometimes passes and sometimes fails even though the application has not changed.

Example:

Run 1 → PASS

Run 2 → FAIL

Run 3 → PASS

Run 4 → FAIL

Flaky tests reduce confidence in the automation suite because engineers cannot easily determine whether a failure is caused by a real defect or by an unstable test.

Issues Identified in the Given Tests

Issue 1: No Wait After Clicking the Login Button

Observation

The test clicks the login button and immediately verifies the dashboard URL.

```
page.click("#login-btn")
```

```
assert page.url ==  
"https://app.workflowpro.com/dashboard"
```

Why this is a problem

After clicking Login, the application performs several background operations such as:

- User authentication
- Session creation

- Token validation
- API requests
- Dashboard loading

These operations take time. If the assertion executes before navigation completes, the test may fail even though the application is functioning correctly.

Recommended Fix

Wait for navigation before checking the URL.

```
page.wait_for_url("**/dashboard")
```

or

```
expect(page).to_have_url("**/dashboard")
```

Issue 2: Dynamic Dashboard Loading

Observation

The test immediately checks whether the welcome message is visible.

```
assert page.locator(".welcome-message").is_visible()
```

Why this is a problem

The dashboard loads dynamically. The welcome message may appear a few seconds after navigation, especially when network latency is high or the dashboard loads multiple widgets.

Recommended Fix

Use Playwright's automatic waiting.

```
expect(page.locator(".welcome-message")).to_be_visible()
```

Issue 3: Hardcoded Credentials

Observation

The email address and password are directly written inside the test.

```
page.fill("#email", "admin@company1.com")  
page.fill("#password", "password123")
```

Why this is a problem

Hardcoded credentials make the test difficult to maintain.

If credentials change, every test must be updated.

Different environments (Development, QA, Staging, Production) usually require different users.

It is also a security risk to store passwords inside source code.

Recommended Fix

Store credentials in configuration files or environment variables.

Example:

EMAIL=admin@company1.com

PASSWORD=*****

The test should read these values during execution.

Issue 4: Browser Configuration

Observation

The browser is launched without specifying execution mode.

```
browser = p.chromium.launch()
```

Why this is a problem

The case study states that the framework should support multiple browsers and integrate with BrowserStack. CI pipelines generally execute tests in headless mode across

different browsers. A test that only works in Chromium may fail in Firefox or Safari.

Recommended Fix

Run tests in headless mode in CI and configure browser selection through the framework.

```
browser = p.chromium.launch(headless=True)
```

Issue 5: Multi-Tenant Loading

Observation

The second test immediately reads all project cards after login.

```
projects = page.locator(".project-card").all()
```

Why this is a problem

The assignment mentions that different tenants have different loading times. Some companies may have hundreds of projects, while others have only a few. Reading project cards before they finish loading can cause intermittent failures.

Recommended Fix

Wait until the project list is visible before validating the content.

Issue 6: No Error Logging or Screenshots

Observation

If the test fails, only the assertion failure is reported.

Why this is a problem

Without screenshots, browser traces, or logs, debugging failures becomes difficult, especially in CI/CD pipelines.

Recommended Fix

Configure the framework to capture:

- Screenshot on failure
- Playwright trace
- Browser console logs
- Network logs

These artifacts help identify the root cause of failures more quickly.

Issue 7: No Retry Strategy

Observation

The test fails immediately if a temporary network issue occurs.

Why this is a problem

Network instability or temporary BrowserStack issues may cause occasional failures that are not related to application defects.

Recommended Fix

Use a limited retry mechanism only for transient failures while ensuring genuine defects are still reported.

Why Does the Test Fail in CI/CD but Pass Locally?

Several factors make CI/CD environments different from local development machines:

- CI servers often have lower CPU and memory resources.
- Network latency is generally higher.
- Tests execute in headless mode.
- Different browsers and screen sizes are used.
- Multiple tests may run in parallel.
- Dynamic loading becomes more noticeable under load.

These conditions expose synchronization issues that may not be visible during local execution.

Improved Login Test

```
from playwright.sync_api import sync_playwright, expect
```

```
BASE_URL = "https://app.workflowpro.com"
```

```
EMAIL = "admin@company1.com"
```

```
PASSWORD = "password123"
```

```
def test_user_login():
```

```
    with sync_playwright() as p:
```

```
        browser = p.chromium.launch(headless=True)
```

```
        page = browser.new_page()
```

```
        page.goto(f"{BASE_URL}/login")
```

```
        page.fill("#email", EMAIL)
```

```
        page.fill("#password", PASSWORD)
```

```
        page.click("#login-btn")
```

```
page.wait_for_url("**/dashboard")
```

```
expect(page.locator(".welcome-message")).to_be_visible()
```

```
browser.close()
```

Additional Improvements

To further improve reliability and maintainability, I would also:

- Implement the **Page Object Model (POM)** to separate page interactions from test logic.
- Store environment-specific values (URLs, credentials, tenant IDs) in configuration files.
- Capture screenshots and traces automatically on failures.
- Execute tests across Chrome, Firefox, and Safari using BrowserStack.
- Use pytest fixtures for browser setup and teardown.
- Generate HTML or Allure reports for easier analysis.
- Use unique test data for parallel execution to avoid conflicts.

Assumptions

For this solution, I made the following assumptions:

- Test users already exist in the system.
- The login endpoint is functioning correctly.
- Test accounts do not require manual 2FA unless specifically configured.
- The dashboard URL is `/dashboard`.
- Each tenant is authorized to view only its own projects.

Conclusion

The original tests demonstrate the intended functionality but are vulnerable to intermittent failures because they rely on immediate assertions and static assumptions about application behavior. By introducing proper synchronization, better configuration management, improved logging, and framework best practices, the automation becomes significantly more stable and maintainable across local execution and CI/CD pipelines.

Part 2 – Test Automation Framework Design

Introduction

The application under test is a multi-tenant B2B SaaS platform that supports multiple browsers, mobile devices, different user roles, API testing, BrowserStack integration, and CI/CD pipelines. To ensure the framework remains maintainable and scalable as the application grows, I would design a modular automation framework using the **Page Object Model (POM)** combined with **pytest fixtures**, **Playwright**, and **API testing**.

The primary goals of the framework are:

- Maintainability
 - Scalability
 - Reusability
 - Easy configuration
 - Cross-browser support
 - Multi-tenant support
 - Easy CI/CD integration
-

1. Framework Structure

I would organize the framework as follows:

automation-framework/

|

|— config/

| config.yaml

| environments.py

|

|— pages/

| login_page.py

| dashboard_page.py

| project_page.py

|

|— tests/

| ui/

| api/

| integration/

```
|
|
|— data/
|   users.json
|   projects.json
|
|
|— utils/
|   logger.py
|   api_client.py
|   browser_factory.py
|
|
|— reports/
|
|
|— screenshots/
|
|
|— conftest.py
|
|
|— requirements.txt
|
```

└─ pytest.ini

Why This Structure?

Each folder has a specific responsibility.

Folder	Purpose
config	Stores environment configurations and URLs
pages	Implements the Page Object Model
tests	Contains UI, API, and Integration tests
data	Stores reusable test data
utils	Helper classes such as logging, API clients, and browser utilities

reports Test execution reports

screenshots Failure screenshots

conftest.py Shared pytest fixtures

requirements.txt Python dependencies

This structure keeps the project organized and easy to maintain.

2. Page Object Model (POM)

I would use the Page Object Model because it separates page interactions from test logic.

Instead of writing selectors inside every test, each page has its own class.

Example:

Login Page



Enter Email



Enter Password



Click Login

Example:

```
class LoginPage:
```

```
    def login(self,email,password):
```

```
self.email.fill(email)
```

```
self.password.fill(password)
```

```
self.login_button.click()
```

Benefits:

- Less duplicate code
- Easier maintenance
- Reusable methods
- Better readability

If a button locator changes, only one file needs to be updated.

3. Configuration Management

The framework should support multiple environments.

Example:

Development

QA

Staging

Production

Instead of hardcoding URLs,
I would use a configuration file.

Example

environment: QA

base_url: https://qa.workflowpro.com

browser: chrome

headless: true

Credentials should be stored using environment variables instead of source code.

Example

EMAIL

PASSWORD

API_TOKEN

This improves both security and maintainability.

4. Multi-Tenant Support

The application supports multiple companies.

Example

Company 1

Company 2

Company 3

Each tenant has:

- Different users
- Different projects
- Different permissions

Therefore, tenant-specific data should be stored separately.

Example

data/

company1_users.json

company2_users.json

The framework should allow selecting the tenant at runtime.

5. User Role Management

The platform supports multiple user roles.

Example:

Admin



Manager



Employee

Each role has different permissions.

Instead of creating different tests,

I would reuse the same tests with different user data.

Example

```
@pytest.mark.parametrize(  
    "user",
```

```
["admin","manager","employee"]  
)
```

This reduces duplicate test cases.

6. API Testing

API testing should be separated from UI testing.

Folder:

tests/api

Benefits:

- Faster execution
- Easy validation
- Independent of UI

API utilities would be placed inside

utils/api_client.py

Common methods:

- GET
- POST
- PUT
- DELETE

This improves code reuse.

7. BrowserStack Integration

The assignment requires BrowserStack support for multiple browsers and mobile devices.

Architecture:

Developer



GitHub



CI Pipeline



Playwright



BrowserStack



Chrome

Firefox

Safari

Android

iPhone

Benefits:

- Cross-browser testing
- Mobile testing
- No need for physical devices
- Cloud execution

BrowserStack credentials should be stored as environment variables.

8. Test Data Management

Test data should never be hardcoded.

Instead,

Create JSON files.

Example

```
{
```

```
"name": "Test Project",
```

```
"description": "Automation Project"  
}
```

Benefits:

- Easy maintenance
- Reusable
- Environment independent

For dynamic tests,

Generate unique data.

Example

Project_20260804_153020

This avoids duplicate data failures.

9. Logging

Logging helps understand what happened during execution.

Every test should log:

- Browser started

- Login successful
- API response received
- Project created
- Test completed

This makes debugging much easier.

10. Reporting

The framework should automatically generate reports.

Possible options:

- HTML Report
- Allure Report
- JUnit XML

Reports should include:

- Test Status
 - Duration
 - Failure Reason
 - Screenshots
 - Browser Information
-

11. Screenshot Capture

Whenever a test fails,

Automatically capture

- Screenshot
- Browser trace

This greatly reduces debugging time.

Example:

reports/



failed_login.png

12. Parallel Execution

To reduce execution time,

Tests should execute in parallel.

Example

Chrome

Firefox

Safari



Run Together

Benefits:

- Faster CI pipeline
- Better browser coverage

Care must be taken to ensure tests use independent test data to avoid conflicts.

13. CI/CD Integration

The automation framework should integrate with Jenkins or GitHub Actions.

Pipeline:

Developer Pushes Code



GitHub



CI Pipeline



Install Dependencies



Run Playwright Tests



Run API Tests



Generate Reports



Upload Reports



Notify Team

Advantages:

- Early bug detection

- Continuous feedback
 - Automatic regression testing
-

14. Missing Requirements / Questions I Would Ask

The case study intentionally leaves some details unspecified. Before implementing the framework, I would clarify the following:

1. How is user authentication handled (Username/Password, OAuth, SSO, or 2FA)?
2. Are test environments shared or dedicated?
3. How should test data be created and cleaned up?
4. What browsers and versions are officially supported?
5. What are the BrowserStack credentials and target devices?
6. Is parallel execution required in CI/CD?
7. Which reporting tool is preferred (HTML, Allure, JUnit)?
8. What is the expected execution time for the regression suite?
9. Are API rate limits applicable?
10. Should failed tests retry automatically?

These questions help ensure that the framework meets both technical and business requirements.

Conclusion

The proposed framework follows modular design principles and emphasizes maintainability, scalability, and reusability. By using the Page Object Model, centralized configuration, reusable utilities, BrowserStack integration, API separation, and CI/CD support, the framework can efficiently support cross-browser, cross-platform, and multi-tenant testing while remaining easy to extend as the application evolves.

Absolutely! This is the **most important section** because it tests your understanding of **API Testing + UI Testing + Mobile Testing + Multi-Tenant Validation**. The case study asks you to design an integration test that creates a project through the API, verifies it in the web UI, checks it on mobile, and confirms tenant isolation.

You can paste the following directly into your Google Doc.

Part 3 – API + UI Integration Test

Introduction

A modern B2B SaaS application consists of multiple layers such as backend APIs, web UI, mobile applications, databases, and authentication services. Testing each layer independently is important, but validating the complete business workflow provides greater confidence that all components work together correctly.

In this section, I have designed an integration test that combines **API testing, Playwright UI automation, and BrowserStack mobile testing** to validate the complete project creation workflow.

Business Scenario

The business requirement is to validate the complete lifecycle of project creation.

The expected workflow is:

Create Project via API



Verify API Response



Login to Web Application



Verify Project Appears in Dashboard



Open Mobile Application



Verify Same Project Appears



Login with Another Tenant



Verify Project is NOT Visible



Delete Test Data

This approach validates:

- Backend functionality
 - UI functionality
 - Mobile compatibility
 - Security
 - Multi-tenant isolation
-

Why Use API + UI Together?

Using only UI automation makes tests slower because every action is performed through the browser.

Instead, the project is created through the API and verified through the UI.

Advantages:

- Faster execution
- Better reliability
- Reduced UI dependency
- Easier debugging

Test Data Strategy

The framework should generate unique test data for every execution.

Example:

```
import uuid
```

```
project_name =  
f"Automation_Project_{uuid.uuid4().hex[:8]}"
```

Benefits:

- Avoids duplicate project names
- Supports parallel execution
- Eliminates data conflicts

Step 1 – Create Project Using API

The first step is to create a project through the REST API.

```
import requests
```

```
headers = {
```

```
"Authorization": "Bearer <token>",
"X-Tenant-ID": "company1"
}

payload = {
    "name": project_name,
    "description": "Created by automation",
    "team_members": ["user1"]
}

response = requests.post(
    "https://app.workflowpro.com/api/v1/projects",
    headers=headers,
    json=payload
)

assert response.status_code == 201

project_id = response.json()["id"]
```

Validation

Verify:

- Status code is 201 (or 200 depending on API contract)
- Project ID is returned
- Project name matches the request

- Status is Active
-

Step 2 – Verify Project in Web UI

After the project is created successfully, log into the application using Playwright.

```
from playwright.sync_api import expect
```

```
page.goto(BASE_URL)
```

```
login_page.login(email, password)
```

```
page.wait_for_url("**/dashboard")
```

```
expect(page.locator(f"text={project_name}")).to_be_visible()  
(
```

Validation

Verify:

- Dashboard loads successfully
 - Newly created project is displayed
 - Project name matches API response
-

Step 3 – Mobile Validation (BrowserStack)

The case study requires mobile testing using BrowserStack. The same project should be visible on supported mobile devices.

Devices:

- Android
- iPhone

Execution Flow:

BrowserStack



Launch Android Device



Login



Open Dashboard



Verify Project Exists

The same validation is repeated for an iPhone device.

Step 4 – Tenant Isolation Testing

Security is extremely important in a multi-tenant application.

A project created under Company 1 must never be visible to Company 2.

Test Flow:

Login as Company 1



Project Visible



Logout



Login as Company 2



Project NOT Visible

Playwright Validation:

```
expect(  
page.locator(f"text={project_name}")  
)not_to_be_visible()
```

This verifies that tenant isolation is working correctly.

Step 5 – Cleanup

Automation should always clean up the data it creates.

Delete the project using the API.

```
requests.delete(  
f"/api/v1/projects/{project_id}",  
headers=headers  
)
```

```
assert response.status_code == 204
```

Benefits:

- Prevents test environment pollution
 - Keeps regression environments clean
 - Supports repeated executions
-

Complete Test Flow

Start Test



Generate Unique Project Name



Create Project via API



Validate API Response

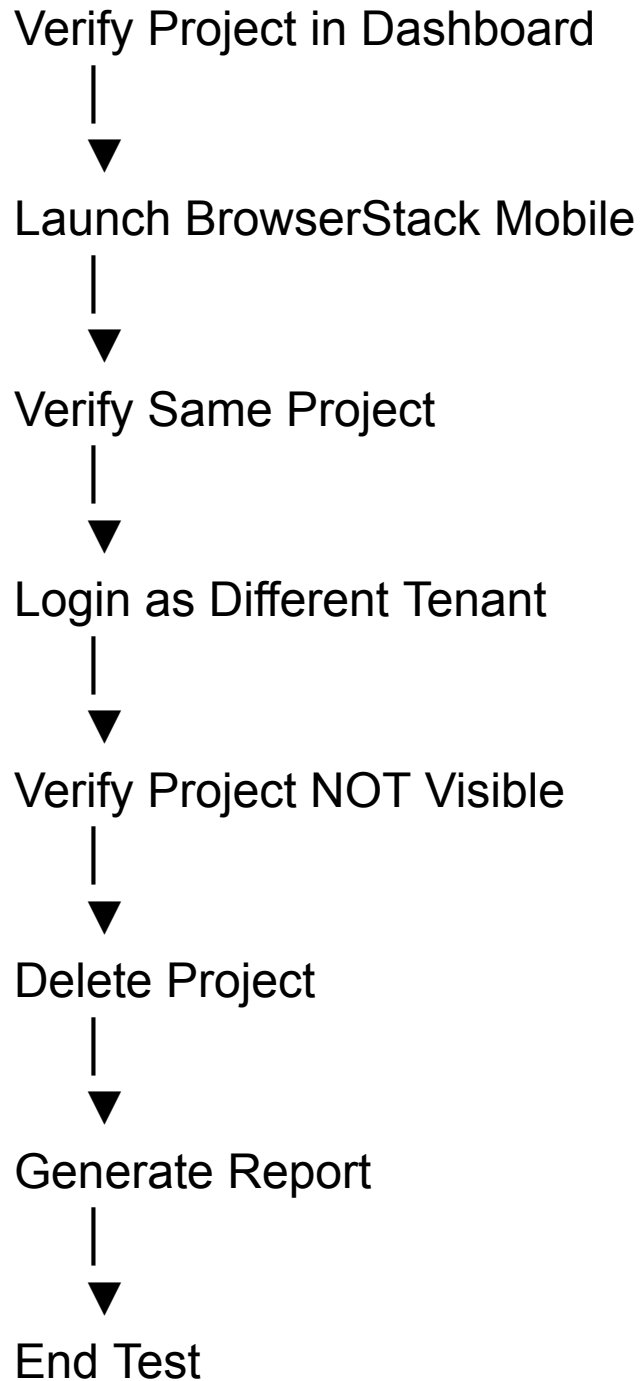


Open Browser



Login





Edge Cases to Validate

In addition to the happy path, I would include the following scenarios:

API

- Invalid authentication token
- Missing tenant ID
- Duplicate project name
- Empty project name
- Invalid request body
- Server error (500)
- Timeout handling

UI

- Slow dashboard loading
- Browser refresh after creation
- Project search functionality
- Empty project list
- Session expiration
- Responsive layout on different screen sizes

Mobile

- Different screen resolutions
 - Portrait and landscape orientation
 - Slow network simulation
 - Touch interactions
 - Mobile responsiveness
-

Error Handling Strategy

The framework should handle unexpected situations gracefully.

Examples:

- Retry API requests for temporary network failures.
 - Wait for page elements using Playwright's built-in expectations.
 - Capture screenshots and browser traces on failure.
 - Log request and response details for failed API calls.
 - Record BrowserStack session links for debugging.
-

Reporting

Each test execution should generate a report containing:

- Test name
- Execution time
- Browser
- Device
- Tenant
- Status (Pass/Fail)
- Screenshots
- API response
- Failure reason (if applicable)

This information helps developers quickly identify and resolve issues.

Assumptions

The following assumptions were made because these details were not provided in the case study:

- A valid API authentication token is available.
 - Test users already exist for Company 1 and Company 2.
 - BrowserStack credentials are configured through environment variables.
 - A DELETE endpoint is available for cleanup.
 - The API returns the project ID after successful creation.
 - Mobile and web applications use the same backend data.
-

Conclusion

This integration test validates the complete project creation workflow across backend APIs, the web interface, and mobile devices while ensuring proper tenant isolation. By combining API testing, Playwright UI automation,

BrowserStack mobile execution, and automated cleanup, the test provides fast, reliable, and comprehensive validation of a critical business workflow. The proposed approach is scalable, maintainable, and suitable for execution within a CI/CD pipeline.
